

143. Sinus, cosinus, une seule table

Pas de virgule flottante, pas de type négatif : une table gravée y suffit.

1. Ce qu'il faut comprendre

Il n'y a ni `sin()` ni `cos()` ici, et pas de type signé non plus — impossible d'écrire un nombre négatif directement. La solution tient dans **une table gravée** : chaque case vaut `40 + 40 * sin(angle)`, toujours entre 0 et 80, jamais en dessous de zéro.

`cos(angle) = sin(angle + 90°)` — et 90°, c'est un quart de tour. Avec trente-deux pas pour un tour complet, un quart de tour fait huit pas. Pas besoin d'une deuxième table : `cosinus` relit la MÊME, huit cases plus loin, avec `%` pour revenir au début quand ça déborde.

Le reste est déjà connu : `aAvant` pour agir **au front** — un seul basculement par appui, et non un par image —, et deux textes de même longueur, pour que rien ne traîne de l'un à l'autre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **693 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	69 o	← la console, à l'allumage — la mise en route de la console
main()	171 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	144 o	← CopierTuiles — 8 lettres sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins
"A : COSINUS"	11 o	← texte() — un texte gravé
"A : SINUS "	11 o	← texte() — un texte gravé
SINUS_0	32 o	← main() — une table gravée

Lettres gravées : ACINOSU:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite() × 1    23 o sur place, dans main()
texte()  × 2    22 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 1    12 o sur place, dans main() + LireManette (37 o)
image()  × 1     6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : une balle traverse l'écran en ondulant (une vague « sinus »).
// A bascule entre sinus et cosinus (la même vague, décalée d'un quart de tour).
//
// Pas de sin() ni de nombres négatifs ici : on grave une TABLE de 32 valeurs,
// un tour complet. Chaque case vaut 40 + 40 × sin(angle) : entre 0 et 80.
// Le cosinus relit la même table, 8 cases plus loin (32 / 4 = un quart de tour).

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <bouton>   // lit un bouton de la manette
#include <texte>    // écrit un texte à l'écran
#include <sprite>   // place un lutin de 8 × 8 au pixel près

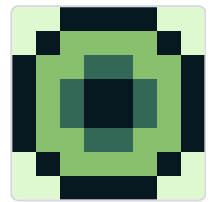
```

2. UN DESSIN — BALLE

```

Tuile BALLE = { "..####..", ".#----#.", "#--++--#", "#-+++-#-", "#-+++-#-", "#--++--#", ".#----#.",
"..####.." };

```



à l'écran

3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — SINUS

```

const uint8_t SINUS[] = {           // 32 pas pour un tour :
    40, 48, 55, 62, 68, 73, 77, 79, // ça monte de 40 à 79...
    80, 79, 77, 73, 68, 62, 55, 48, // ... sommet à 80, puis ça redescend...
    40, 32, 25, 18, 12, 7, 3, 1,    // ... sous 40...
    0, 1, 3, 7, 12, 18, 25, 32,     // ... creux à 0, et ça remonte.
};

```

4. UNE VARIABLE — X

```

uint8_t x = 0;           // La balle, en pixels (de gauche à droite).

```

5. UNE VARIABLE — ANGLE

```

uint8_t angle = 0;       // Où l'on en est dans la table : 0 à 31.

```

6. UNE VARIABLE — COSINUS

```

uint8_t cosinus = 0;     // 0 = sinus, 1 = cosinus.

```

7. UNE VARIABLE — AAVANT

```

uint8_t aAvant = 0;      // A à l'image d'avant.

```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    while (true) {                          // Répète pour toujours :
        image();                            // attend l'image suivante.

        uint8_t a = bouton(A);
        if (a && !aAvant) cosinus = !cosinus; // Début d'appui : on bascule (0 ↔ 1).
        aAvant = a;

        if (cosinus) texte(1, 1, "A : COSINUS");
        else texte(1, 1, "A : SINUS  ");      // Deux espaces : même longueur que COSINUS.

        uint8_t indice = cosinus ? (angle + 8) % 32 : angle;
                                // Cosinus : 8 cases plus loin ; % 32 revient au début
                                // si on dépasse (angle 30 → 38 % 32 = 6). Sinus : angle.
        uint8_t y = 28 + SINUS[indice];        // La hauteur : entre 28 et 108.

        sprite(0, x, y, BALLE); // On pose la balle.

        x++;                                // Un pixel à droite...
        if (x > 152) x = 0;                 // ... et au bord, on repart de la gauche.

        angle++;                            // Le pas suivant de la vague...
        if (angle >= sizeof(SINUS)) angle = 0; // ... et après 31, on revient à 0.
    }

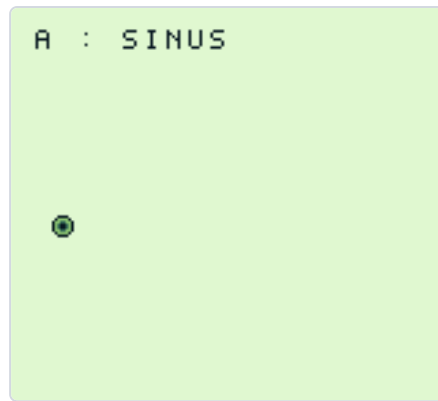
    return 0;                               // Jamais atteint.
}
```

3. Ce qui se passe, étape par étape

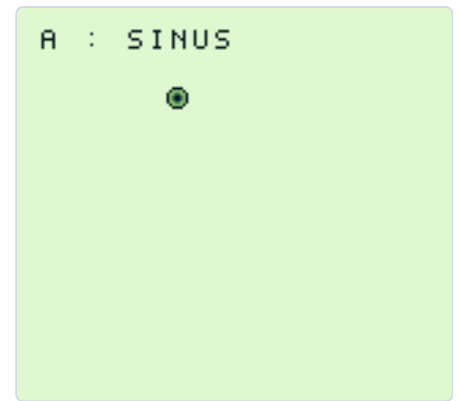
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



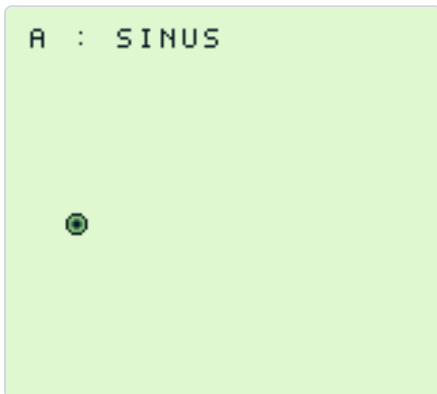
la toute première image — la console vient de démarrer — image 1



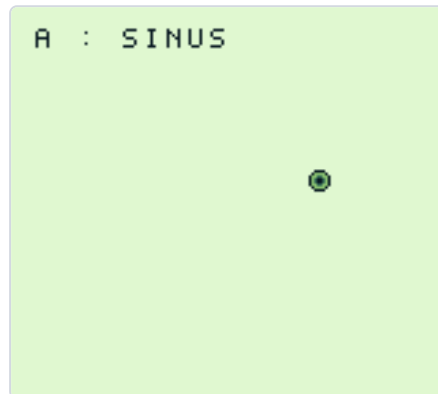
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



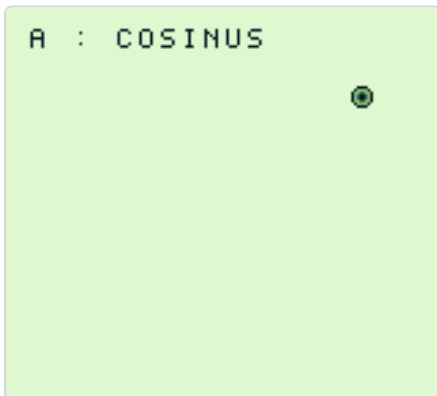
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Une balle qui ondule en traversant l'écran ; A bascule « SINUS » en « COSINUS », et elle saute d'un quart de tour.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre de départ est SINUS
- ✓ elle porte la tuile dessinée
- ✓ elle reste dans la plage de la table (min 28, max 108)
- ✓ elle ondule vraiment, et ne file pas tout droit
- ✓ A bascule sur COSINUS, au front
- ✓ le titre change, à la même longueur

Ensuite : **143.1. Chapitre 25 — Tes propres #include : ajouter une fonction : ce qui vient** — chapitre 25 : Tes propres #include : ajouter une fonction. · [gameboy3](#) — écrire une cartouche Game Boy en C++